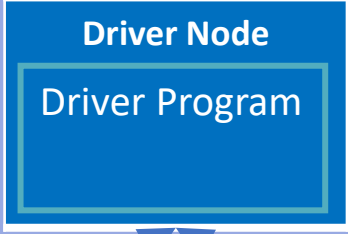


Performance Tuning – Data Skewness Issue



Network

Worker Node 1

Executor

1	10	val1	val2	val3	val4	val5
1	100	val1	val3	val4	val4	val5
1	20	val1	val4	val5	val4	val5

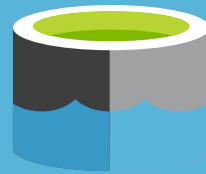
2	200	val1	val2	val3	val4
---	-----	------	------	------	------

Worker Node 2

Executor

3	30	val1	val6	val7	val4	val5
3	300	val1	val7	val8	val4	val5

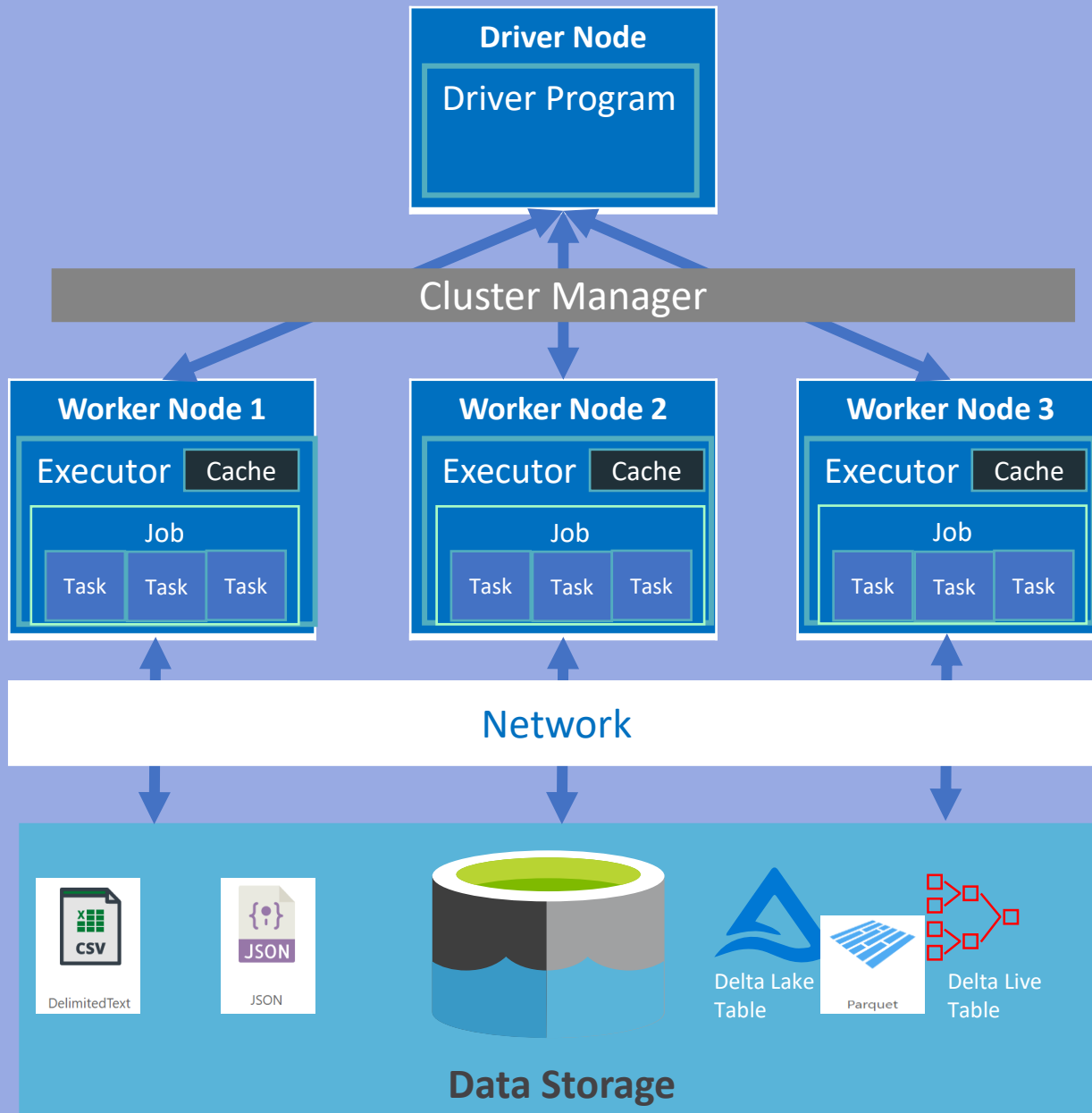
4	40	val1	val8	val9	val4	val5
---	----	------	------	------	------	------



Data Storage

Key	value	col1	col2	col3	col4	col5
1	10	val1	val2	val3	val4	val5
1	100	val1	val3	val4	val4	val5
1	20	val1	val4	val5	val4	val5
2	200	val1	val5	val6	val4	val5
3	30	val1	val6	val7	val4	val5
3	300	val1	val7	val8	val4	val5
4	40	val1	val8	val9	val4	val5

Performance Tuning – Data Skewness(Uneven Data Distribution) Issue



Only one stage is taking longer time even after all remaining stages completed

Normally occurs when column values counts got large disparity

Shuffle Read Size/Records in Spark UI have large difference between min & max

Use SQL hints to specify columns and values causing Skewness

```
SELECT /*+ SKEW('table', 'column_name', (value1, value2)) */ * FROM table
```

ADAPTIVE QUERY EXECUTION(AQE) automatically solve the Skewedness issue by setting the property(Default true)

```
set spark.sql.adaptive.skewJoin.enabled = false
```

Partition has 245MB of data and Five times larger than average partition size then AQE consider that as Skewed Partition and use the properties and values to find tune AQE

```
set spark.sql.adaptive.skewJoin.skewedPartitionFactor = <value> (default 5)
```

```
set spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes = <size in bytes> (default 256 MB)
```

Use Salting as the last option to solve skewed partitions

In Salting breaking the largest skewed partitions into smaller partitions by changing skewed column values appended with random integers.